

SENAI – ESCOLA HERMENEGILDO CAMPOS DE ALMEIDA CURSO
TÉCNICO EM CIBERSISTEMAS PARA AUTOMAÇÃO – 3º SEMESTRE

EDUARDA ISIDORIO DA SILVA
GILBERTO ALVES DE MELO RAMOS
JÚLIA LOPES DA SILVA
PEDRO ARTHUR BARBANCHO SANTOS

DOCUMENTAÇÃO TÉCNICA
JEGP CORPORATION – PROJETO INTEGRADOR

GUARULHOS – SP
2026

EDUARDA ISIDORIO DA SILVA
GILBERTO ALVES DE MELO RAMOS
JÚLIA LOPES DA SILVA
PEDRO ARTHUR BARBANCHO SANTOS

DOCUMENTAÇÃO TÉCNICA
JEGP CORPORATION – PROJETO INTEGRADOR

Documentação apresentada como uma junção de todas as disciplinas aplicadas a um só projeto integrador do curso técnico em Cbersistemas para automação como requisito parcial para aprovação semestral, sob a orientação do prof. William Belarmino da Silva.

GUARULHOS-SP
2026

SUMÁRIO

1. INTRODUÇÃO.....	1
1.1. Organização da Equipe.....	1
1.2. Configuração do Ambiente.....	1
1.3. Investigação dos pinos.....	2
1.4. Perguntas.....	3
1.5. Fluxograma.....	3
CONCLUSÃO.....	4
REFERÊNCIAS	

1. INTRODUÇÃO AO PROJETO

Neste projeto vamos atuar como uma equipe de desenvolvimento de uma empresa de automação industrial no qual será responsável por criar um sistema inteligente de monitoramento utilizando o microcontrolador ESP8266 e Shield HY-M302 tendo como objetivo nessa primeira etapa garantir a configuração correta do ambiente, que o hardware seja compreendido e validado e o comportamento do sistema bem definido.

1.1 ORGANIZAÇÃO DA EQUIPE

Ao iniciarmos o projeto em um grupo com 4 pessoas que nomeamos como JEGP Corporation separamos 4 cargos onde cada membro ficou com um deles, os cargos são: programador que é responsável por desenvolver o código, analista responsável pela investigação técnica e definição de regras, documentador responsável pelo registro e organização da documentação e o testador responsável por fazer as testagens, identificar falhas e validar o projeto na prática. Apesar dos membros terem uma função definida cada uma delas é rotativa, ou seja, cada um dos integrantes vai interagir com as demais funções e exercê-las dando total apoio e ajuda necessária.

As integrantes Eduarda Isidorio e Júlia Lopes ficaram responsáveis principalmente pela documentação técnica e pela análise e os membros Gilberto Alves e Pedro Arthur ficaram responsáveis principalmente pela programação e testagem no projeto.

1.2 CONFIGURAÇÃO DO AMBIENTE

Na configuração do ambiente temos como objetivo configurar o Arduino IDE para a comunicação com o ESP8266. Para configurar passamos pelo procedimento técnico a seguir:

Acesso ao menu Arquivo → Preferências

Inserção da URL:

http://arduino.esp8266.com/stable/package_esp8266com_index.json Acesso ao

Gerenciador de Placas

Instalação do pacote ESP8266

Seleção da placa LOLIN(WEMOS) D1 R2 & mini

Seleção da porta COM

Para fazer a testagem foi utilizado o código Blink para verificar o upload de código, execução na placa e a comunicação serial.

1.3 INVESTIGAÇÃO DOS PINOS

Cuidar para usar os pinos do ESP8266 do jeito certo é o que ajuda a garantir que todo o sistema funcione bem. Desse jeito, a gente consegue evitar que ele não ligue direito, que as peças estraguem ou que ele comece a se comportar de um jeito estranho enquanto está trabalhando.

Os pinos que podem ser utilizados como entrada no ESP8266 são, principalmente, D1, D2, D5, D6 e D7, pois são estáveis e não interferem no funcionamento da placa. Esses são os mais indicados para conectar sensores com segurança. O pino RX também pode funcionar como entrada, mas não é recomendado porque é usado na comunicação com o computador.

O ESP8266 D1 tem 11 pinos digitais que você consegue usar (do D0 ao D8, e também o RX e TX). A questão é que não dá para usar todos eles como quiser, porque alguns já vêm com funções específicas para quando o aparelho liga (boot) e para a comunicação serial.

Os pinos mais seguros para ser usados são do D1 ao D7 pois funcionam como entrada e saída, não interferem no boot e também não tem função crítica ativa o que seria o uso ideal para sensores (DHT11), botões, LEDs e relés, de forma resumida são os pinos mais confiáveis. Já os pinos nos quais devemos usar com mais cuidado (já que podem dar algum problema) são o D3, D4 e D8 que participam do boot da placa. Quando ligamos o ESP8266 ele olha estes pinos para decidir como iniciar. Se eles estiverem no estado errado a placa não vai ligar direito ou vai entrar em um modo errado, um exemplo é:

O D3 precisa estar HIGH (ligado)

O D8 precisa estar LOW (desligado)

Caso ligar um sensor ali que puxa o sinal errado dará algum problema. Achemos importante ressaltar que se pode usar sim, porém tem que saber exatamente o que está fazendo.

Também há os pinos que devem ser evitados como o RX e TX que são usados para comunicação com o computador e upload de código, o problema aqui é que se usar estes pinos pode atrapalhar o upload, pode dar conflito com o sistema ou então os dados ficam misturados. E por fim temos um pino muito limitado que é o D0 que não funciona com interrupções e tem limitações internas, ele funciona mas não serve pra muitas coisas.

Resumindo: Os pinos do ESP8266 não são todos iguais; alguns têm mais restrições. Isso acontece porque eles participam tanto do processo de inicialização (o boot) quanto de funções internas do microcontrolador. Se usar esses pinos de forma errada, corre o risco de o aparelho não ligar direito, de ter problemas de comunicação, ou de o sistema ficar instável.

PINO	CÓDIGO	Pode usar?	TIPO	RESTRIÇÃO	JUSTIFICATIVA
D2	D0 – GPIO16	SIM	ENTRADA	NENHUMA	FUNCIONA PERFEITAMENTE
D3	D1 – GPIO 05	SIM	ENTRADA	NENHUMA	FUNCIONA PERFEITAMENTE
D4	D2 – GPIO 04	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
D6	D3 – GPIO 0	NÃO RECOMENDADO	SAÍDA	NENHUMA	PODE AFETAR NA INICIALIZAÇÃO DO ESP8266
D9	D4 – GPIO 2	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
D13& D5	D5 – GPIO 14	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
D12	D6 – GPIO 12	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
D10	D7 – GPIO 13	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
D11	D8 – GPIO 15	SIM	SAÍDA	NENHUMA	FUNCIONA PERFEITAMENTE
NONE	RX – GPIO 3	SIM	DESCONHECIDO	NENHUMA	COMUNICAÇÃO SERIAL
NONE	TX – GPIO 1	SIM	DESCONHECIDO	NENHUMA	COMUNICAÇÃO SERIAL
A0	A0 – GPIO 0	SIM	DESCONHECIDO	NENHUMA	FUNCIONA PERFEITAMENTE

1.4 PERGUNTAS

1. Nem todos os pinos podem ser usados. Por quê?

Nem todos os pinos podem ser usados por que alguns possuem funções específicas

Durante a inicialização da placa (boot), enquanto outros são utilizados para comunicação. E existem pinos com limitações elétricas ou funcionais, e devido a isso nem todos podem ser utilizados sem causar problemas no funcionamento do sistema.

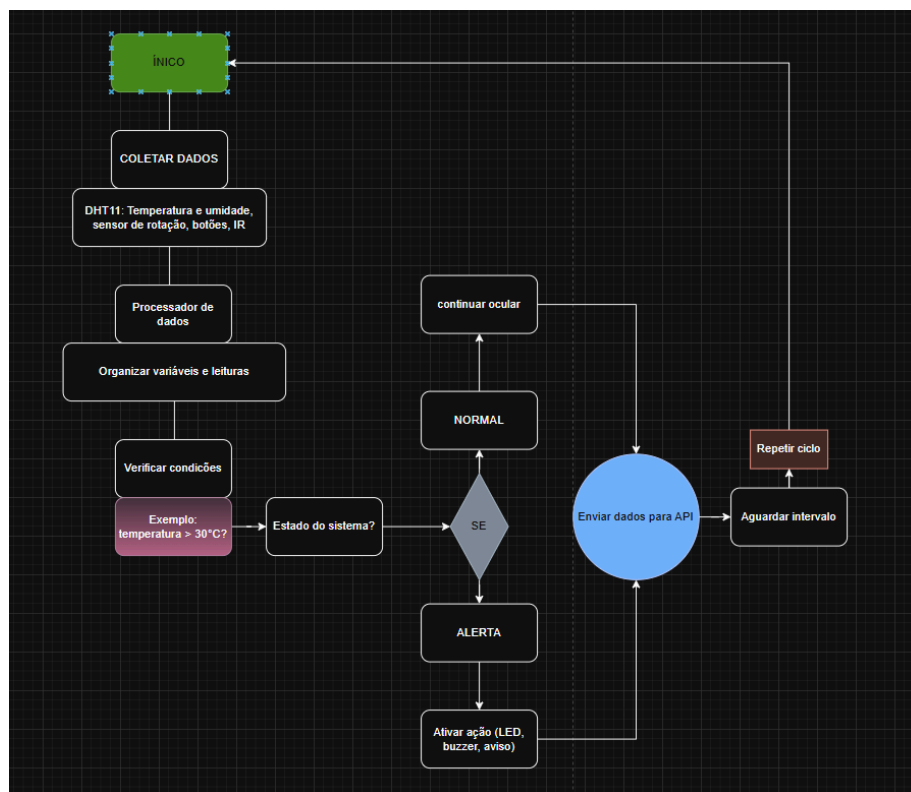
2. O que pode acontecer se um pino inadequado for utilizado?

O que pode acontecer é a placa não iniciar corretamente, podendo acabar entrando em modo de programação, apresentar falhas ou executar comportamentos inesperados.

3. Como isso impacta um sistema real de automação?

Isso impacta muito em um sistema real de automação como, falhas críticas, interrupções no funcionamento e até risco de segurança dependendo da aplicação.

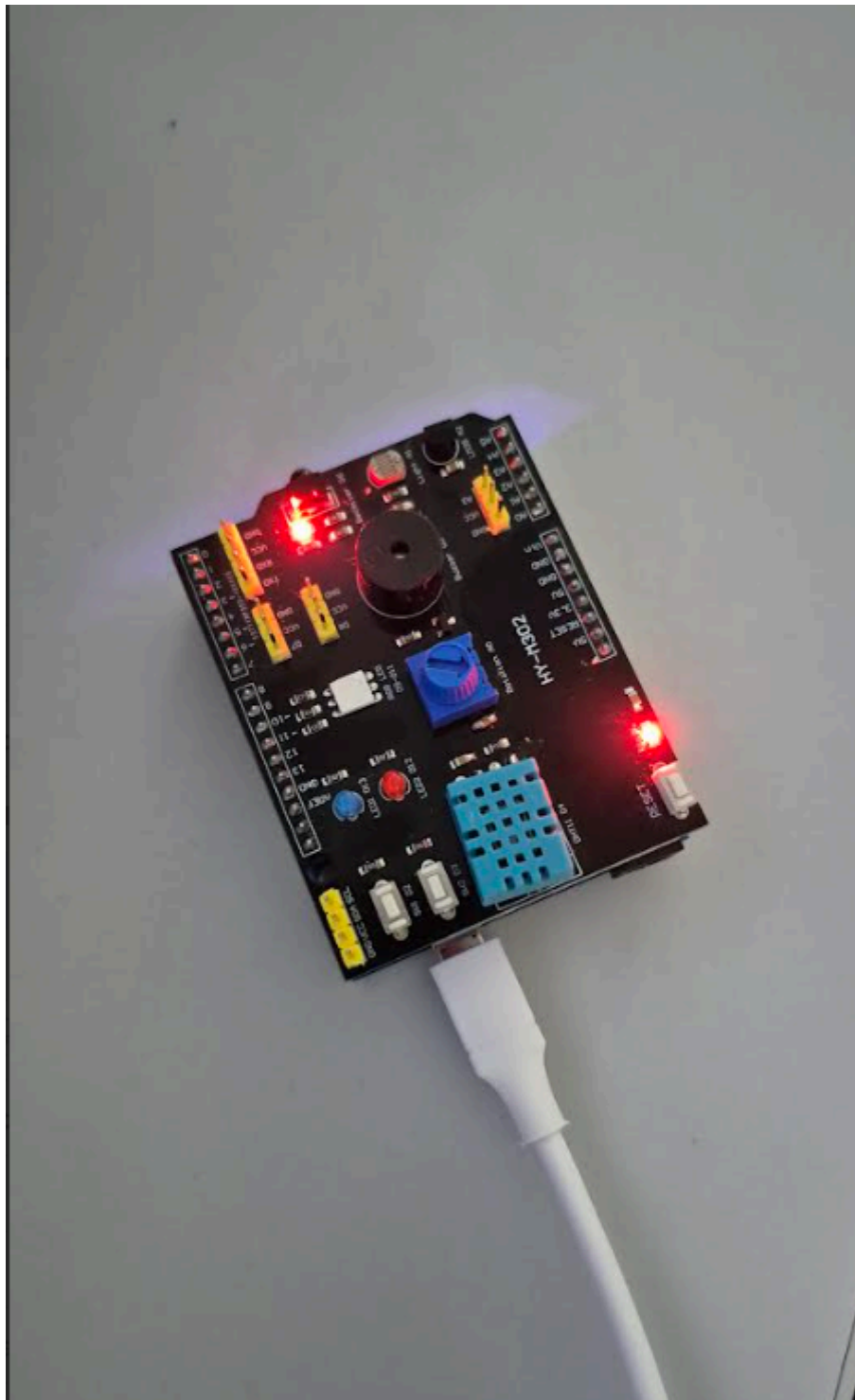
1.5 FLUXOGRAMA



```

1  int buzzer = D5;
2  int buzzer2 = D6;
3  int botao = D0;
4  int botao2 = D1;
5  int analogico = A0;
6
7  int vermelho = D4;
8  int azul = D7;
9  int verde = D8;
10
11 void setup() {
12     Serial.begin(9600);
13
14     pinMode(buzzer, OUTPUT);
15     pinMode(buzzer2, OUTPUT);
16     pinMode(vermelho, OUTPUT);
17     pinMode(azul, OUTPUT);
18     pinMode(verde, OUTPUT);
19
20     pinMode(botao, INPUT_PULLUP);
21     pinMode(botao2, INPUT_PULLUP);
22 }
23
24 void loop() {
25     int valor = analogRead(analogico);
26     Serial.print("Valor A0: ");
27     Serial.println(valor);
28
29     // --- LÓGICA DAS CORES (Potenciômetro / Sensor) ---
30
31     // Primeiro apagamos todos para não misturar as cores
32     digitalWrite(vermelho, LOW);
33     digitalWrite(azul, LOW);
34     digitalWrite(verde, LOW);
35
36     if (valor > 400) {
37         digitalWrite(verde, HIGH); // Maior que 400: VERDE
38     }
39     else if (valor > 250) {
40         digitalWrite(azul, HIGH); // Entre 250 e 400: AZUL
41     }
42     else if (valor > 100) {
43         digitalWrite(vermelho, HIGH); // Entre 100 e 250: VERMELHO
44     }
45     // Se for menor que 100, todos continuam apagados
46
47     // --- LÓGICA DOS BOTÕES ---
48
49     if (digitalRead(botao) == LOW) {
50         digitalWrite(buzzer, HIGH);
51     } else {
52         digitalWrite(buzzer, LOW);
53     }
54
55     if (digitalRead(botao2) == LOW) {
56         digitalWrite(buzzer2, HIGH);
57     } else {
58         digitalWrite(buzzer2, LOW);
59     }
60
61     delay(100);
62 }

```

CONCLUSÃO

Nesta primeira etapa do projeto, a equipe JEGP Corporation conseguiu organizar as funções dos integrantes, configurar corretamente o ambiente de desenvolvimento da Arduino IDE e validar a comunicação com o ESP8266 através do teste Blink.

Também foi realizado um estudo dos pinos da placa, identificando quais são mais seguros para uso e quais exigem cuidados por influenciarem no boot e na comunicação serial. Esse entendimento é importante para evitar falhas, garantir estabilidade e melhorar o funcionamento do sistema.

Com isso, o projeto possui uma base sólida para avançar para as próximas etapas de desenvolvimento do sistema inteligente de monitoramento.

REFERÊNCIAS

<https://embarcados.com.br/arduino-entradas-analogicas/> Acessado dia 23 de Abril de 2026

<https://embarcados.com.br/pinos-digitais-do-arduino/> Acessado dia 23 de Abril de 2026

<https://youtu.be/SYKx85uoBrw> Acessado dia 23 de Abril de 2026

<https://youtu.be/yehyUmUDJXc> Acessado dia 23 de Abril de 2026

https://www.usinainfo.com.br/shieldsexpansores/shield-multifuncoes-hy-m302-para-arduino-com-dht11-lm35-receptor-ir-ldr-leds-buzzer-e-outros-4866.html?srsId=AfmBOop_2HPLcnKXZyT8dFzBrSbnXadyb1m1YbbANW4K7eJ6r4pqGXZ4 Acessado dia 23 de Abril de 2026